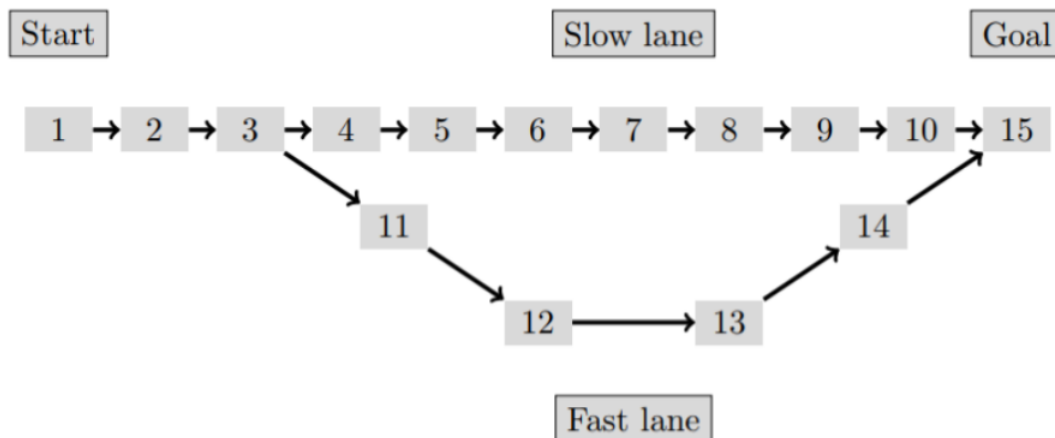

LINFO2275

First project

Group 2:

Gavin Hope 09022500 – SINF
Jakob Thibodeau 12752500 – DATI
Lore Lernoux 77562000 – MAP



Introduction

In this project, we present the implementation of a Markov Decision Process to solve a Snakes and Ladders game. To do so, we implemented the value iteration algorithm to find the optimal policy for the game. This policy specifies, for each tile, the best dice to use in order to minimize the expected number of turns to reach the goal (tile 15). Indeed, the player may choose between four different dices at each turn, each with some probabilities of moves and traps triggering. There are two game configurations considered: one circular (must stop on tile 15 exactly) and a linear (must only reach tile 15). Afterwards, we compared this optimal policy to other suboptimal policies, such as fixed-dice policies, to see how they perform in terms of expected number of turns to win the game. Both theoretical results from value iteration and empirical results from Monte Carlo simulations are analyzed, presented and compared across different trap layouts, for both game configurations.

Theoretical background

The game is modeled as a Markov Decision Process. As seen during the lectures, it is defined by

- a finite set of states S , which in our case corresponds to the tiles of the board, i.e. $S = \{1, 2, \dots, 15\}$;
- a set of actions A , which corresponds to the choice of dice to roll, i.e. $A = \{1, 2, 3, 4\}$;
- a transition function $P(s' | s, a)$, which gives the probability of reaching state s' from state s after choosing action a . These probabilities are determined by the stochastic outcomes of the dice, the branching structure of the board, and the activation of traps;
- a cost function $c(s | a)$, which gives the cost when taking action a in state s . In our case, we can define the cost as 1 for each turn taken, and 0 when reaching the goal (tile 15).

The goal is to find an optimal policy π^* specifying, at each tile, which dice minimizes the expected number of turns to reach tile 15 from any initial state. Let $V(s)$ be this total expected number of turns which respects Bellman's equation with as condition $V(15) = 0$:

$$V(s) = \min_{a \in A} \left[c(s | a) + \sum_{s' \in S} P(s' | s, a) V(s') \right] \quad (1)$$

This optimal expected number of turns can be computed using the value iteration algorithm, which iteratively updates $V(s)$ for each state until convergence. The optimal policy is then obtained by selecting, for each state, the action that minimizes the Bellman equation.

To validate the theoretical results, we performed Monte Carlo simulations. To do so, we simulated 1000 games from the initial state until the goal state following a strategy, and we computed the empirical average number of turns, $\bar{T}$. We also computed the standard error of these estimates based on the sample standard deviation $\hat{\sigma}$.

$$\bar{T} = \frac{1}{N} \sum_{i=1}^N T_i$$

$$SE = \frac{\hat{\sigma}}{\sqrt{N}}$$

$\bar{T}$ is then compared to the theoretical value $V(1)$ obtained through value iteration in order to validate the correctness of the optimal policy.

Implementation description

Trap layouts

We chose to use 6 different trap layouts to compare the optimal policy in different settings. These layouts are the following :

- **No traps**: this layout has no traps, which means that the player can only move forward and never go back. This is the simplest layout and serves as a baseline for comparison.
- **Few traps**: this layout has a few traps teleporting the player three squares backward scattered throughout the board.

- **Many traps:** this layout has many traps of all types scattered throughout the board, which makes it more difficult to win the game.
- **Evil fast lane:** this layout has restart and backward traps only on the fast lane.
- **Two-in-a-row traps:** this layout presents two segments with two consecutive backward trap tiles, making it more difficult to avoid them.
- **Back-to-three traps:** this layout has traps that send the player back to tile 3, which is the branching tile.

Sub-optimal strategies

We chose to implement five sub-optimal strategies to compare with the optimal policy. These strategies are the following :

- **One dice only:** for each of the 4 dices, this policy simply chooses one specific dice. This gives us 4 sub-optimal policies to compare with the optimal one.
- **Uniform random strategy:** this policy chooses a dice uniformly at random, independently of the current tile. This is a baseline strategy.

Results

Optimal Policy

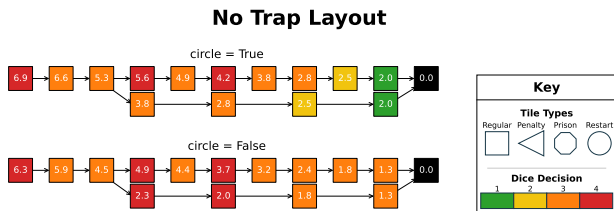


Figure 1: Expected number of turns to win the game from each tile for the no traps layout. The tile color represents the optimal policy obtained through value iteration.

Figure 1 shows the optimal policy in the case where no traps are present, in both the circular and linear game structures.

For tiles closer to the goal, the circle structure favors safer dice as we must stop exactly on the goal, while the linear structure favors faster dices as we can overshoot the goal. Indeed, since moving back to tile 1 is always a bad outcome, which is the case in overshoot, the optimal policy will always choose dice 1 (the safe dice) on both possible last tiles when the layout is circular, regardless of the trap layout.

For the rest of the tiles, the policy mostly favors dice 3, as it has the highest expected value (1.5) of displacement of all dice, when traps are not triggered. This confirms that, when no penalties are present, the best strategy is to take as much risk as possible to get to the goal faster.

Die 4 (the special dice) is chosen on tile 1, since the player cannot move further back, bringing the expected value of the dice to 2. Tiles 4 and 6 also favor the special dice, since there is a $1/4$ chance to move back directly to tile 3 and potentially move to the fast lane. We find it surprising that tile 3 itself does not favor the special dice, as it could directly land on the goal tile.

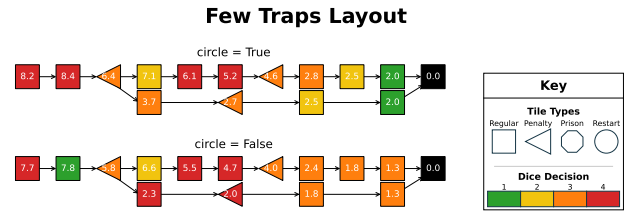


Figure 2: Expected number of turns to win the game from each tile for the few traps layout.

Figure 2 adds a few penalty traps into the layout. We see some recurring patterns like choosing safer dices close to the goal for the looped layout and prioritizing the riskier dices in safer regions. On most trapped tiles, the optimal policy chooses the risky dice even with the $\frac{1}{4}$ chance of activating the trap. This can be explained by comparing the expected displacement: even with the penalty traps which effectively act as -3 rolls when starting on them, the risky dice has an expected value of $\frac{3}{4}$, which is higher than the safe and normal dices

($\frac{1}{2}$). More interestingly, the optimal policy clearly tries to avoid the traps when it can by anticipating them. For example, it chooses the normal dice on tile 4 to get closer to the trapped tile 7 until it is close enough (on tile 5 or 6) to be able to leap over the trap using the special dice.

The non-looped layout prioritizes heavily taking the fast lane, as seen by the choice of the safe dice on tile 2. Even if taking the fast lane is not guaranteed, if there is even a slight chance of going there, it takes it. In practice, the player will only rarely take the fast lane with this policy since the only way to end on tile 2 is to roll a -3 when standing on tile 5. Still, when it happens, it is a huge speedup.

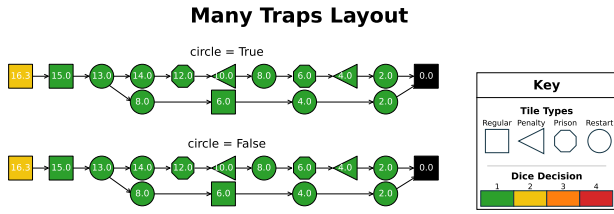


Figure 3: Expected number of turns to win the game from each tile for the many-trap layout.

As see in Figure 3, having many traps unsurprisingly leads to the near exclusive use of the safe dice. As the safe dice moves the player by at most 1 tile, having the layout loop or not doesn't affect the optimal policy choice or its effectiveness. This is further shown by comparing to the only dice 1 strategy, which takes almost the same number of turns as the optimal strategy.

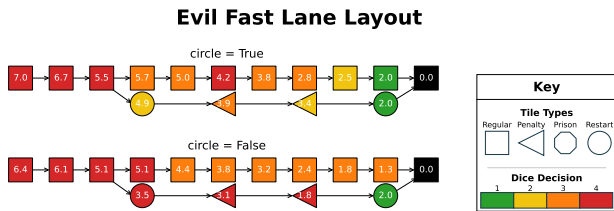


Figure 4: Expected number of turns to win the game from each tile for the evil fast lane layout.

When designing the *evil fast lane* layout seen in Figure 4, we expected the optimal policy to avoid

the fast lane at all costs. However, that is not what we observe, as the choice of the special dice at tile 7 for the looped layout and tile 4 for the non-looped layout may directly lead the player to land on tile 3 and merge into the fast lane. This is an acceptable outcome for the model, since rolling a 5 with the special dice from tile 3 would lead directly to the goal tile when taking the fast lane.

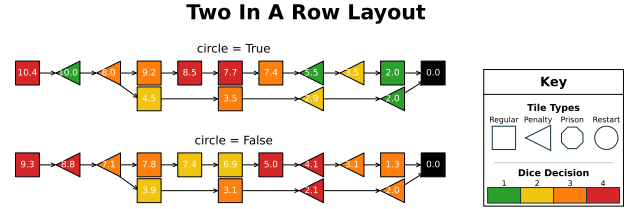


Figure 5: Expected number of turns to win the game from each tile for the two-in-a-row traps layout.

In the layout shown in Figure 5, we observe a drastic difference in strategy between the looped and unlooped boards. In the looped board, the player tries to leap the traps, but only from tiles that guarantee it can land on or before the goal if it rolls a 5, as shown by the choice of the special dice on tiles 5 and 6. However, in the unlooped layout, the player prefers inching closer using the normal dice, even risking the activation of the trap on tiles 8, 13 and 14, so that a 3 or a 5 on the special dice makes it reach the goal (skipping over the 2 traps in a row).

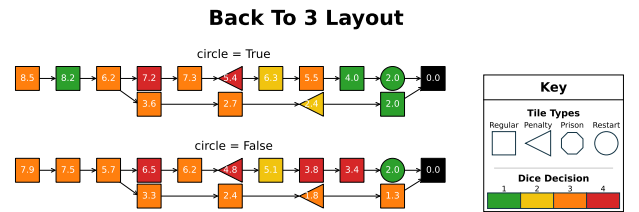


Figure 6: Expected number of turns to win the game from each tile for the back-to-three traps layout.

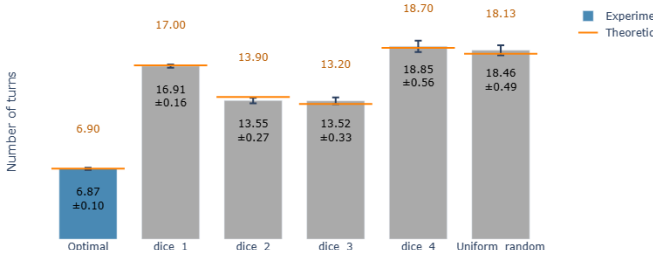
In the back-to-3 layout shown in Figure 6, the optimal strategy becomes much more sophisticated. In the looped layout, the player would

much rather take the fast lane (as evidenced by the choice of the safe dice on tile 2) since it avoids having to tiptoe past the restart trap on tile 10. Surprisingly, the optimal strategy chooses the risky dice at tile 8, risking triggering the restart trap. It appears that the risk of restarting is worth taking over moving slowly (with the safe dice) to the goal. The player in the non-looped layout does not benefit as much from the fast lane, as it can simply leap over tile 10 to reach the goal with the special dice.

Theoretical vs Empirical Results: Sub-optimal Strategies

We compared the optimal policy with the 5 sub-optimal strategies by simulating 1000 games for each policy and game structure (circle or not) and computing the average number of turns to win the game, i.e. the expected number of turns needed to reach tile 15 from tile 1. For each trap layout, we represented the game and the optimal policy by coloring the tiles according to the dice that should be used on that tile.

Number of Turns for no_trap_layout (circle) by Strategy, Expected Value vs Average turns in 1000 games



Number of Turns for no_trap_layout (not circle) by Strategy, Expected Value vs Average turns in 1000 games

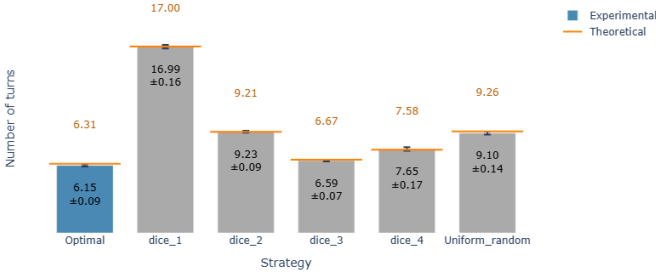
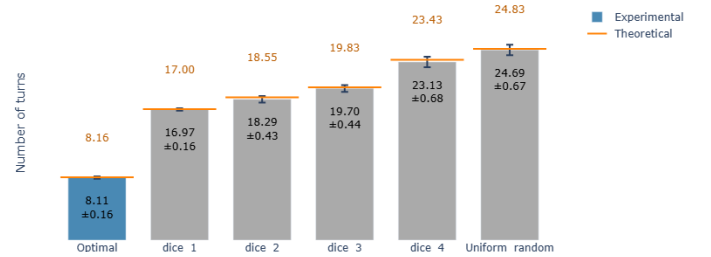


Figure 7: Expected vs Average simulated number of turns to win by strategy for No Traps Layout

Figure 7 compares different strategies to the optimal policy in the no-traps layout. In the circle layout, the optimal policy far outperforms the

other simpler strategies, since using riskier dice first and safer dice near the goal brings a huge advantage to the player. For the non-circle layout, the advantage is less significant, since repeatedly rolling dice 3 is nearly the optimal policy anyway. Note that for all layouts, looped or not, the *safe dice only* strategy has an expected number of turns of 17. This value remains the same, as the safe dice never triggers any traps and can move the player at most 1 tile, never overstepping the goal. We can see that the experimental results for all strategies are very close to the theoretical expected number of turns, which validates our results and the correctness of our implementation of value iteration.

Number of Turns for few_traps_layout (circle) by Strategy, Expected Value vs Average turns in 1000 games



Number of Turns for few_traps_layout (not circle) by Strategy, Expected Value vs Average turns in 1000 games

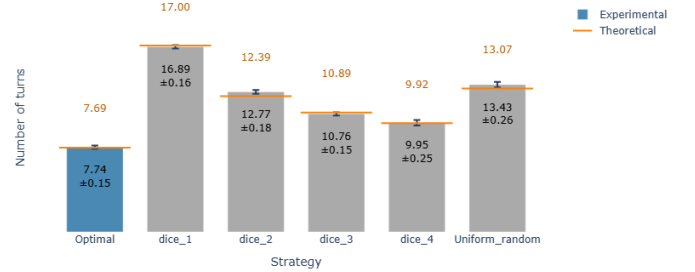
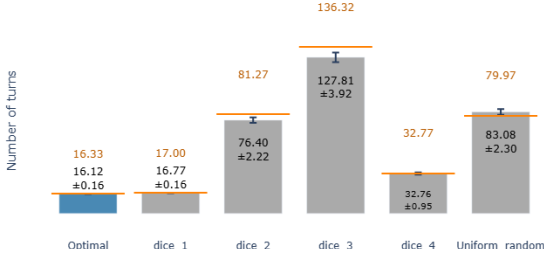


Figure 8: Expected vs Average simulated number of turns to win by strategy for Few Traps Layout

Comparing the optimal policy to other strategies for the few-trap layout, we observe mostly the same trends as with no traps. However, since the use of riskier dice is now punished, the strategies solely using them are worse, digging the optimal policy's lead even further. Indeed, a more heterogeneous use of the different dice is required to get a good performance, as the player needs to be able to adapt to the traps and use the safe dice when needed, while still taking risks when it is worth it. The uniform random strategy has worse performance than the optimal policy, but

close to all other strategies, which was expected since it has a $\frac{1}{4}$ chance of using the each dice at each tile.

Number of Turns for many_traps_layout (circle) by Strategy, Expected Value vs Average turns in 1000 games



Number of Turns for many_traps_layout (not circle) by Strategy, Expected Value vs Average turns in 1000 games

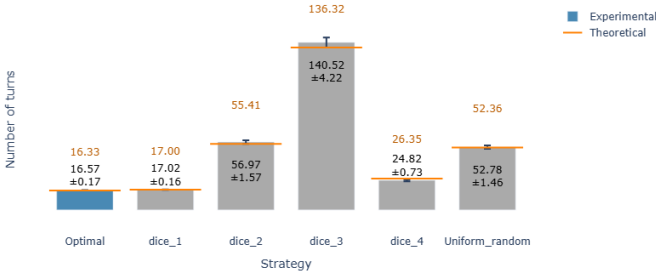
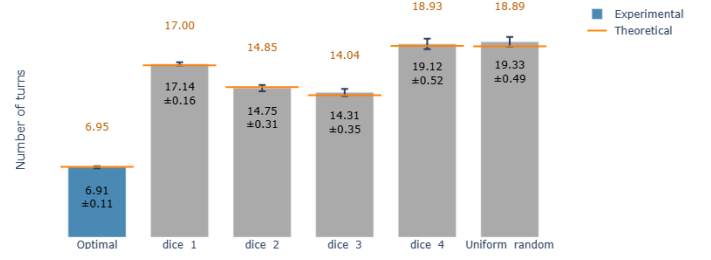


Figure 9: Expected vs Average simulated number of turns to win by strategy for Many Traps Layout

As expected, since the optimal policies mainly use the safe dice, its performance is comparable to the *safe dice only* strategy. We can indeed see that the theoretical number of turns are the same in both configurations, as indicated earlier. All other strategies, especially the *risky dice only* strategy, are penalized heavily by the traps. The *special dice only* strategy performs a little bit better, however, since it can leap over multiple traps in a single roll, and the traps do not activate in the same pass. Choosing a dice uniformly at random performs better than the risky dice only strategy, since it has a $\frac{1}{4}$ chance of using the safe dice, but it is still much worse than the optimal policy, which uses almost exclusively the safe dice.

Number of Turns for evil_fast_lane_layout (circle) by Strategy, Expected Value vs Average turns in 1000 games



Number of Turns for evil_fast_lane_layout (not circle) by Strategy, Expected Value vs Average turns in 1000 games

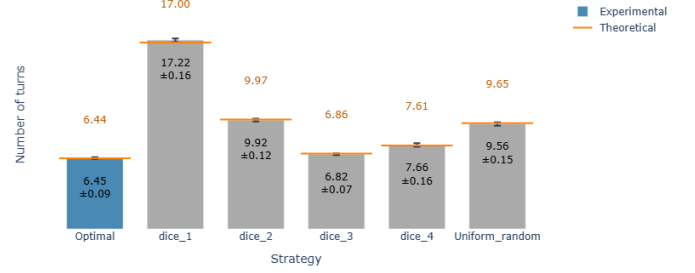
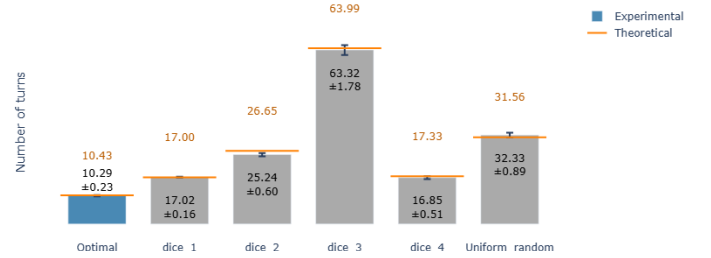


Figure 10: Expected vs Average simulated number of turns to win by strategy for evil fast lane Layout

The comparisons in Figure 10 continue to show the dominance of the optimal strategy, with the number of turns taken by all strategies closely following that of the no-trap layout.

Number of Turns for two_in_a_row_layout (circle) by Strategy, Expected Value vs Average turns in 1000 games



Number of Turns for two_in_a_row_layout (not circle) by Strategy, Expected Value vs Average turns in 1000 games

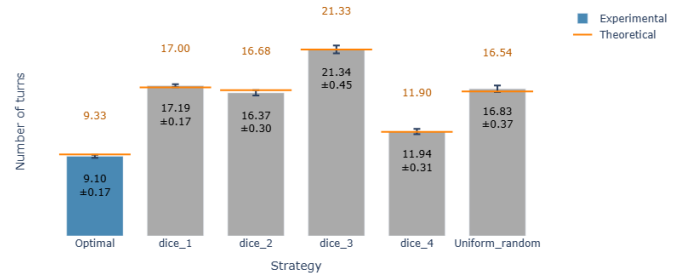


Figure 11: Expected vs Average simulated number of turns to win by strategy for two-in-a-row Layout

Looking at the comparisons in Figure 11, we find that the risky dice is heavily penalized if mis-used, since the chance of trap activation is much higher. Surprisingly, for the looped version, the "only special dice" strategy performs better on this layout than on the no-trap layout. This is due to the trap on tile 9, which if activated, leads the player back to tile 6, from which a roll of 5 directly lands on the goal. Compare that to the no-trap layout, where landing on tile 9 would mean a roll of 3 or 5 overshoots the goal and a roll of -1 or -3 moves the player back (like a trap) but consumes an extra turn (unlike a trap).



Figure 12: Expected vs Average simulated number of turns to win by strategy for back-to-three Layout

Again, the optimal policy outperforms all other strategies. Having the layout loop or not clearly affects how risk-averse the player can be, as the risky dice and the special dice policies are nearly twice and thrice as fast, respectively.

Conclusion

Overall, value-iteration was able to consistently provide the least number of turns compared to other more naive strategies. It was able to accomplish this with, at least for this relatively

small problem, usually below 50 iterations. In problems with simple models such as this one, a tighter stability tolerance is reasonable given the fast per-iteration calculation time. For problems with a high cost per iteration, either computationally or monetarily, a larger stability tolerance could be chosen. By selecting a larger stability tolerance, the number of iterations can be decreased in exchange for a more suboptimal but quicker to determine policy. This makes value-iteration a flexible tool for dealing with MDPs of all problem-sizes. Another factor to consider is problem-complexity. For simple cases such as "many-traps" the naive solution was able to perform nearly as well as the optimal strategy, with no value-iteration step required. In either case, the value-iteration policy still performed the best of any of the other strategies tested.